

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΣΧΟΛΗ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

Τεχνολογία Λογισμικού

Εισαγωγή στην Τεχνολογία Λογισμικού.

Διδάσκων
Δρ Ιωάννης Στύλιος

Δομή Μαθήματος

ΔΙΑΛΕΞΗ & ΕΡΓΑΣΤΗΡΙΟ

- **ΔΙΑΛΕΞΗ (ΔΕΥΤΕΡΑ, 10.00-13.00)**
- Μέρος 1: Τεχνολογία Λογισμικού & Σχεδίαση
- Μέρος 2: Προγραμματισμός & Υλοποίηση

- **ΕΡΓΑΣΤΗΡΙΟ (ΔΕΥΤΕΡΑ, 13.00 - ???)**
- Εφαρμογή θεωρίας στη πράξη
- Ασκήσεις σχεδιασμού & προγραμματισμού

Δομή Μαθήματος

ΠΕΡΙΕΧΟΜΕΝΟ ΜΑΘΗΜΑΤΟΣ

ΤΕΧΝΟΛΟΓΙΑ ΛΟΓΙΣΜΙΚΟΥ

- Κύκλος ζωής λογισμικού
- Μοντέλα ανάπτυξης
- Ανάλυση και τεκμηρίωση απαιτήσεων
- Διαγράμματα UML
- Αντικειμενοστρεφής σχεδίαση

ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΣΕ KOTLIN

- Βασική σύνταξη και δομές ελέγχου και επαναληπτικές δομές
- Συναρτήσεις και δομές δεδομένων
- Κλάσεις και αντικείμενα

Δομή Μαθήματος

ΜΑΘΗΣΙΑΚΟΙ ΣΤΟΧΟΙ

- Να μπορείτε να επιλέγετε το κατάλληλο μοντέλο ανάπτυξης για διαφορετικά έργα
- Να μπορείτε να αναλύετε και να τεκμηριώνετε απαιτήσεις λογισμικού αποτελεσματικά
- Να μπορείτε να σχεδιάζετε λογισμικό χρησιμοποιώντας διαγράμματα UML
- Να μπορείτε να εφαρμόζετε αρχές αντικειμενοστρεφούς προγραμματισμού στην πράξη
- Να μπορείτε να μετατρέπετε σχεδιαστικά διαγράμματα σε λειτουργικό κώδικα
- Να μπορείτε να αξιολογείτε και να βελτιώνετε την ποιότητα του λογισμικού

Αξιολόγηση

Εργασία - Εξέταση

Δομή Βαθμολογίας: Το μάθημα περιλαμβάνει δύο προόδους, οι οποίες είναι απαλλακτικές για όσους πετύχουν προβιβάσιμο βαθμό. Η κατανομή του βαθμού διαμορφώνεται ως εξής:

- Πρώτη πρόοδος: 50%
- Δεύτερη πρόοδος: 20% ή 50%
- Εργασία: 30%

Εναλλακτική Επιλογή:

- Όσοι συμμετέχουν στις προόδους αλλά δεν υποβάλουν εργασία ο τελικός βαθμός προκύπτει από τις 2 προόδους.
- Όσοι δώσουν μόνο μια πρόοδο το υπόλοιπο 50% ή 20% (αν δώσουν εργασία) θα δοθεί στην τελική εξέταση.
- Όσοι δεν συμμετάσχουν στις προόδους και στην εργασία, η τελική βαθμολογία θα προκύψει αποκλειστικά από την τελική εξέταση.

Εξέταση Εργαστηρίου (Προαιρετική):

- Η εξέταση του εργαστηρίου είναι προαιρετική και μπορεί μόνο να ενισχύσει τον τελικό βαθμό σας.

- Οι εξετάσεις: με ανοιχτές σημειώσεις μαθήματος.

- Κατοχύρωση:

– Ο βαθμός από τις εργασίες κατοχυρώνεται μέχρι και το Σεπτέμβριο.
– Τίποτε δεν κατοχυρώνεται για επόμενη χρονιά!

Συνεργατική Μάθηση στην Τεχνολογία Λογισμικού

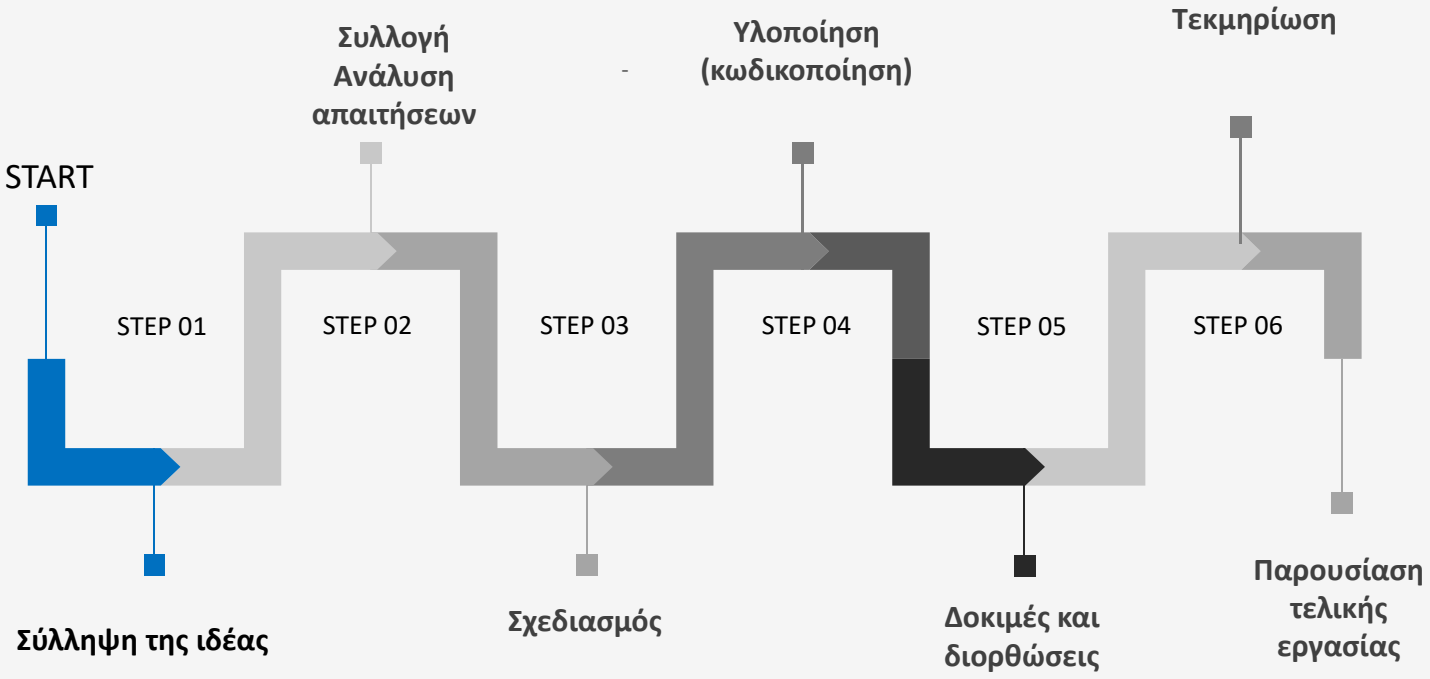
Ομαδικό project ανάπτυξης λογισμικού

- Ενίσχυση της ομαδικής εργασίας μέσα από ένα κοινό project.
- Οι ομάδες αναλύουν απαιτήσεις, σχεδιάζουν, υλοποιούν και παρουσιάζουν μια εφαρμογή λογισμικού.
- Ανάθεση και εναλλαγή ρόλων:
  1. Αναλυτής απαιτήσεων
  2. Σχεδιαστής λογισμικού (UML)
  3. Προγραμματιστής (Kotlin)
  4. Υπεύθυνος δοκιμών και ποιότητας
  5. Υπεύθυνος τεκμηρίωσης και συντονισμού



Εργασία

Παραδοτέα



ΤΟ ΠΑΙΧΝΙΔΙ ΤΩΝ ΕΡΩΤΗΣΕΩΝ:

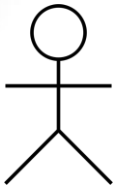
ΟΛΑ ΟΣΑ ΘΕΛΕΤΕ ΝΑ ΡΩΤΗΣΕΤΕ ΜΑ ΔΕΝ ΤΟΛΜΑΤΕ!



ΕΘΕΛΟΝΤΕΣ;

ΜΕΡΟΣ Α’

ΤΕΧΝΟΛΟΓΙΑ ΛΟΓΙΣΜΙΚΟΥ



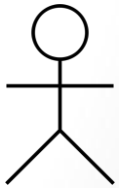
ΤΙ ΕΙΝΑΙ ΤΟ «ΛΟΓΙΣΜΙΚΟ», ΠΟΥ ΤΟΥΣ ΚΑΝΕΙ ΟΛΟΥΣ ΝΑ ΜΙΛΟΥΝ ΓΙΑ ΑΥΤΟ;

Εμένα άκου, θα σου πω!

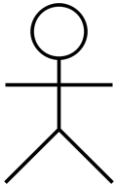
«Είναι ένα σύνολο **προγραμμάτων**, **διαδικασιών** και **οδηγιών**… που εργασίες εκτελούν σε συστήματα υπολογιστών!»

ΚΑΙ… ΤΙ ΠΕΡΙΛΑΜΒΑΝΕΙ;

Πέραν από τον **κώδικα** εννοείς; Περιλαμβάνει τις **βιβλιοθήκες**, τα **δεδομένα** και τις **ρυθμίσεις** που απαιτούνται για τη λειτουργία του. Εννοείται πως συνοδεύεται πάντα από την απαραίτητη **τεκμηρίωση!**

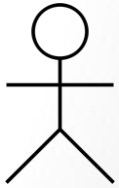


ΚΑΙ ΔΕ ΜΟΥ ΛΕΣ, ΥΠΑΡΧΟΥΝ ΠΟΛΛΑ ΕΙΔΗ ΛΟΓΙΣΜΙΚΟΥ;



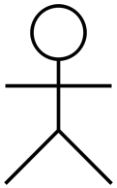
Υπάρχουν. Για παράδειγμα, έχουμε το **λογισμικό συστήματος** (UNIX, DOS, Windows, βιομηχανικών συστημάτων), το **λογισμικό εφαρμογών** (επιχειρηματικές, επιστημονικές, εκπαιδευτικές, ψυχαγωγίας), το **λογισμικό ενσωματωμένο σε συσκευές** (routers, εκτυπωτές) κ.ά.

ΜΑΛΙΣΤΑ! ΤΩΡΑ ΚΑΤΑΛΑΒΑ… ΚΑΙ ΠΩΣ ΦΤΙΑΧΝΟΝΤΑΙ ΟΛΑ ΑΥΤΑ;



Για την ανάπτυξη λογισμικού χρησιμοποιούμε μια ποικιλία εργαλείων όπως **ολοκληρωμένα περιβάλλοντα ανάπτυξης** (Integrated Development Environment), **συστήματα ελέγχου εκδόσεων**, **εργαλεία διαχείρισης έργων**…

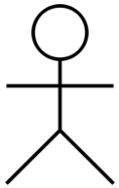
Α, ΩΡΑΙΑ! ΠΑΩ ΝΑ ΦΤΙΑΞΩ ΤΟ ΔΙΚΟ ΜΟΥ ΛΟΓΙΣΜΙΚΟ!



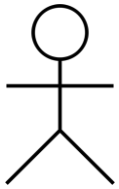
ΔΕΝ ΘΕΛΩ ΝΑ ΣΕ ΑΠΟΓΟΗΤΕΥΣΩ, ΑΛΛΑ ΔΕΝ-ΕΙΝΑΙ-ΤΟΣΟ-ΑΠΛΟ!

Το λογισμικό έχει έναν **κύκλο ζωής** που περιλαμβάνει τη **σύλληψη της ιδέας**, την **ανάλυση απαιτήσεων**, τον **σχεδιασμό**, την **υλοποίηση** (κωδικοποίηση), τη **δοκιμή**, την **κυκλοφορία**, τη **συντήρηση** και τελικά την **απόσυρση**… 🪦



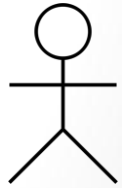


ΜΕ ΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΛΟΓΙΣΜΙΚΟΥ ΤΙ ΠΑΙΖΕΙ; ΑΛΛΟ ΕΙΝΑΙ ΑΥΤΟ;

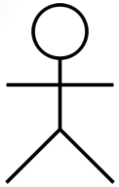


«Είναι **κλάδος της πληροφορικής** που ασχολείται με τη **μελέτη** και την **εφαρμογή** συστηματικών, μεθοδικών, και ποσοτικοποιημένων **προσεγγίσεων** για την **ανάπτυξη**, τη **λειτουργία** και τη **συντήρηση** του λογισμικού». (IEEE Standard 610.12)

ΚΑΙ ΠΟΙΕΣ ΕΙΝΑΙ ΑΥΤΕΣ ΟΙ ΠΡΟΣΕΓΓΙΣΕΙΣ;



Υπάρχουν διάφορες **μεθοδολογίες ανάπτυξης λογισμικού** όπως η μέθοδος καταρράκτης (**waterfall**), το σπειροειδές μοντέλο (**Spiral Model**), το μοντέλο πρωτυποποίησης (**Prototyping model**) κ.ά., που προσφέρουν διαφορετικές προσεγγίσεις στην **οργάνωση** και **διαχείριση** των διαδικασιών.

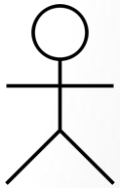


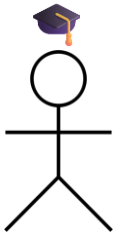
SOUNDS EASY…

ΕΙΝΑΙ ΟΛΑ ΤΟΣΟ ΑΠΛΑ ΚΑΙ ΕΥΚΟΛΑ ΟΣΟ ΤΑ ΠΑΡΟΥΣΙΑΖΕΙΣ; 🤔



Η ανάπτυξη λογισμικού αντιμετωπίζει προκλήσεις όπως η **διαχείριση** της **πολυπλοκότητας**, η **εκτίμηση** του **κόστους** και του **χρόνου**, η **διασφάλιση** της **ποιότητας** και η **προσαρμογή** στις μεταβαλλόμενες **απαιτήσεις** κ.ά.



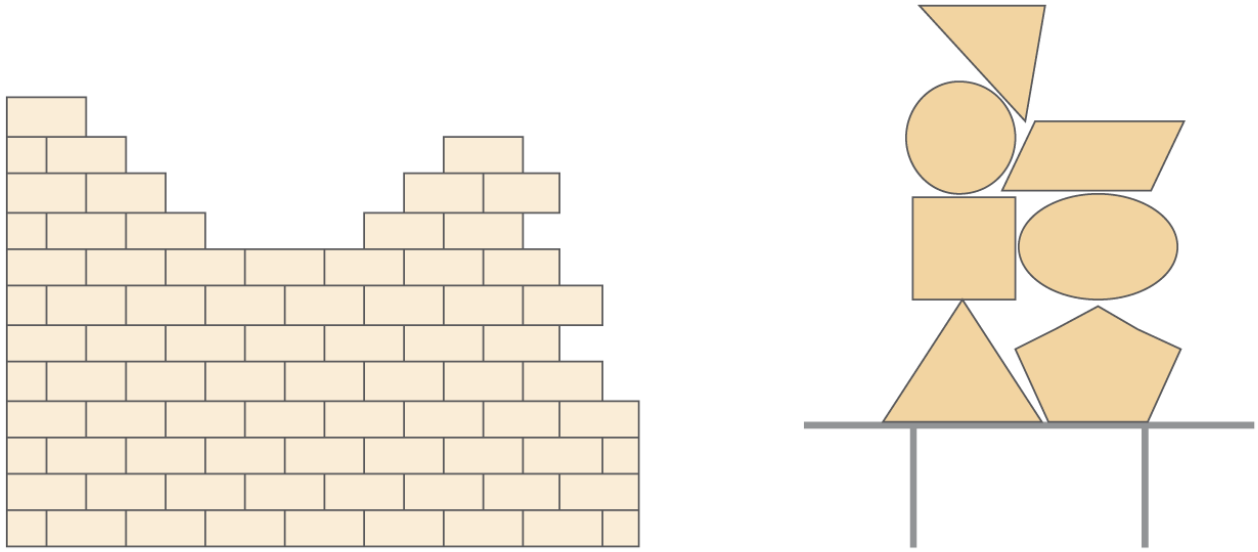


ΜΕΡΟΣ Β’

«ΛΟΓΙΣΜΙΚΟ»



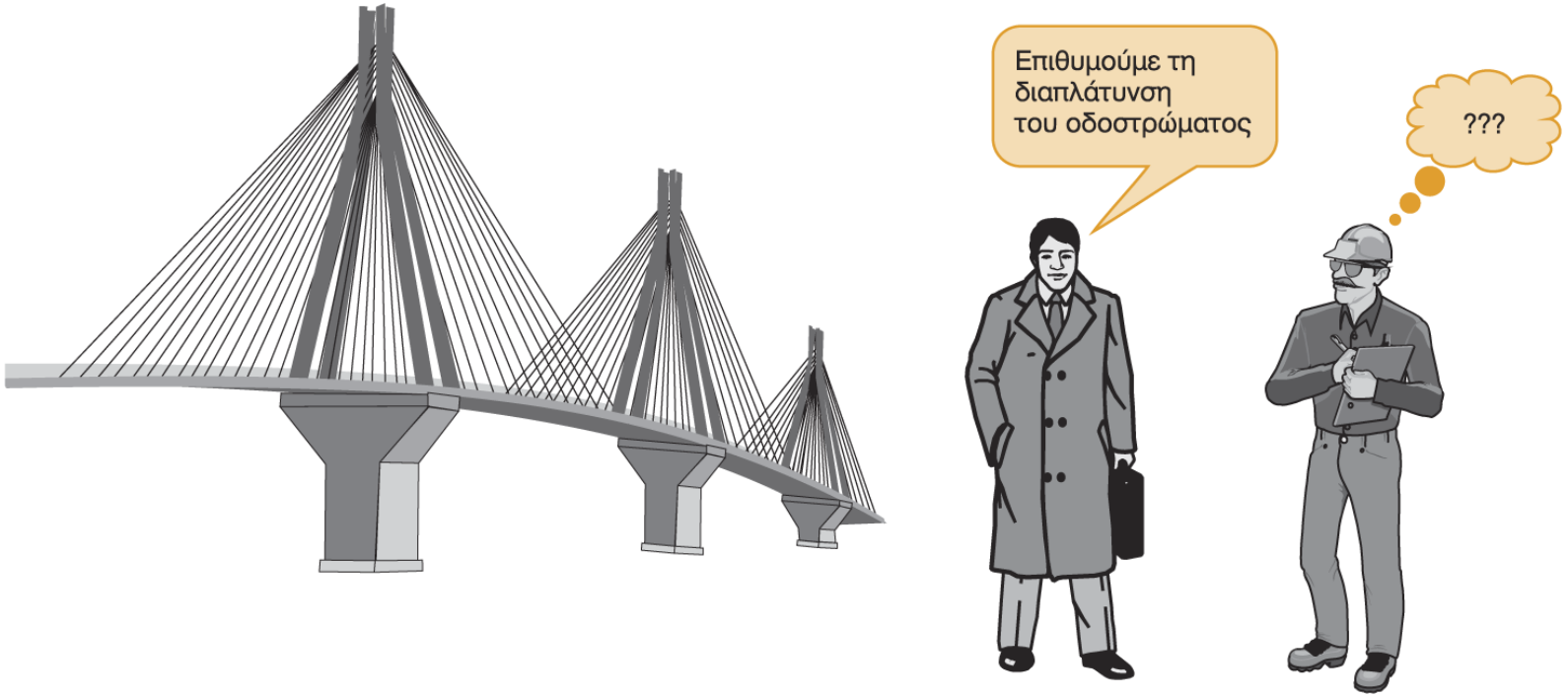
ΛΟΓΙΣΜΙΚΟ – ΠΟΛΥΠΛΟΚΟΤΗΤΑ



Σχήμα 1-4: Το λογισμικό κατασκευάζεται από ανομοιογενή υλικά με σύνθετες διεπαφές σε αντίθεση με άλλες κατασκευές

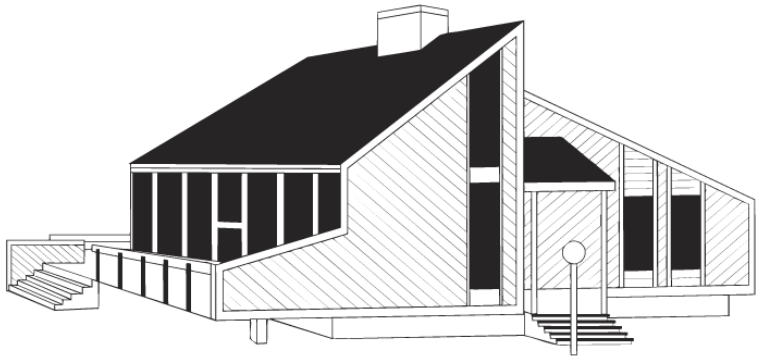
- Η ανάπτυξη λογισμικού απαιτεί πολλά συστατικά (λ.χ., γλώσσες προγραμματισμού, βιβλιοθήκες κώδικα, βάσεις δεδομένων κ.λπ.)
- Τα οποία συχνά συνεπάγονται πολύπλοκες αλληλεπιδράσεις (λ.χ., διασφάλιση συμβατότητας μεταξύ διαφορετικών εκδόσεων, ενσωμάτωση με υπάρχοντα συστήματα, κ.λπ.)
- Τα παραπάνω μας προσφέρουν αρκετή ευελιξία αλλά ενέχουν και πολυπλοκότητα!

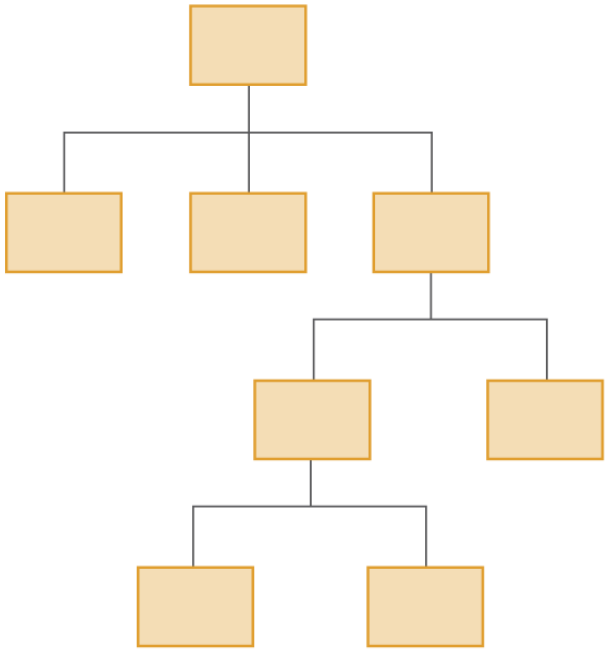
ΛΟΓΙΣΜΙΚΟ – ΠΡΟΣΑΡΜΟΣΤΙΚΟΤΗΤΑ



Σχήμα 1-5: Το λογισμικό αλλάζει και μετά την κατασκευή του

ΛΟΓΙΣΜΙΚΟ – ΜΗ ΦΥΣΙΚΗ ΥΠΟΣΤΑΣΗ





Σχήμα 1-6: Τα μοντέλα του λογισμικού δεν οπτικοποιούν πλήρως τη συμπεριφορά του

Τα μοντέλα είναι πολύτιμα εργαλεία για το σχεδιασμό και την ανάλυση λογισμικού, αλλά η ολοκληρωμένη κατανόηση και η διασφάλιση της ορθής λειτουργίας απαιτούν επίσης ενδελεχή έλεγχο και παρακολούθηση του συστήματος σε πραγματικό περιβάλλον.

«ΚΡΙΣΗ» ΛΟΓΙΣΜΙΚΟΥ – ΖΗΤΗΜΑΤΑ

- Αποκλίσεις του λογισμικού από τη **λειτουργικότητα** που επιθυμούσαν οι πελάτες.
- Αποκλίσεις από το **χρονοδιάγραμμα** ανάπτυξης.
- Αποκλίσεις από τον **οικονομικό προγραμματισμό** ανάπτυξης του λογισμικού.
- Αποκλίσεις από το **προβλεπόμενο κόστος** του **περιβάλλοντος λειτουργίας** του.
- **Αδυναμία εξέλιξης** με σκοπό να προσαρμοσθεί στις νέες ανάγκες των πελατών.

«ΚΡΙΣΗ» ΛΟΓΙΣΜΙΚΟΥ – ΛΟΓΟΙ

- Ανεπαρκής προσδιορισμός απαιτήσεων -> **προβληματική σχεδίαση**
- Μη ρεαλιστικοί στόχοι του έργου -> **ελλιπής αναφορά προόδου**
- Μη ακριβείς εκτιμήσεις απαιτούμενων πόρων -> **ελλιπής χειρισμός ρίσκου**
- Αδυναμία χειρισμού πολυπλοκότητας -> **έλλειψη εμπειρίας με τεχνολογία**
- **Κακή επικοινωνία** μεταξύ των εμπλεκόμενων μερών (!)



ΑΠΑΝΤΗΣΗ ΣΤΗ ΚΡΙΣΗ;

ΤΕΧΝΟΛΟΓΙΑ ΛΟΓΙΣΜΙΚΟΥ

ΠΩΣ ΒΟΗΘΑΕΙ;

ΕΠΙΣΤΗΜΟΝΙΚΗ ΘΕΜΕΛΙΩΣΗ ΤΟΥ ΛΟΓΙΣΜΙΚΟΥ

- ✓ Κύκλος ζωής (από την ιδέα έως την απόσυρση)
- ✓ Διαδικασία παραγωγής (ρόλοι και σειρά εργασιών)
- ✓ Τρόπος περιγραφής – τεκμηρίωσης (απαιτήσεις και αποφάσεις)
- ✓ Διαδικασία συντήρησης (διορθώσεις και προσαρμογές)

ΠΟΙΟΤΙΚΑ ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ ΠΡΟΪΟΝΤΩΝ ΛΟΓΙΣΜΙΚΟΥ (1/2)

- ✓ **Ορθότητα (Correctness):** Ένα πρόγραμμα είναι λειτουργικά ορθό όταν **συμπεριφέρεται σύμφωνα με τις καταγεγραμμένες λειτουργικές απαιτήσεις**.
- ✓ **Αξιοπιστία (Reliability):** Το λογισμικό εκτελεί τις απαιτούμενες λειτουργίες **χωρίς αστοχία** για καθορισμένο χρονικό διάστημα και υπό συγκεκριμένες συνθήκες. *(Υπενθύμιση: σταθερή λειτουργία στον χρόνο.)*
- ✓ **Αποδοτικότητα (Performance):** Το πρόγραμμα δεν θα πρέπει να κάνει **αλόγιστη χρήση των πόρων του συστήματος**.
- ✓ **Ευχρηστία (Usability):** Οι προβλεπόμενοι χρήστες μπορούν να πετυχαίνουν τους στόχους τους **αποτελεσματικά, με εύλογη προσπάθεια και ικανοποίηση.**

ΠΟΙΟΤΙΚΑ ΧΑΡΑΚΤΗΡΙΣΤΙΚΑ ΠΡΟΪΟΝΤΩΝ ΛΟΓΙΣΜΙΚΟΥ (2/2)

- ✓ **Ευελιξία – Συντηρησιμότητά (Maintainability)**: Σε περίπτωση αλλαγής των απαιτήσεων ένα πρόγραμμα **θα πρέπει να μπορεί να εξελίσσεται**.
- ✓ **Ελεγξιμότητα (Verifiability)**: **Οι ιδιότητες** ενός συστήματος λογισμικού **θα πρέπει να μπορούν να ελεγχθούν εύκολα** (λ.χ. με χρήση προσομοίωσης).
- ✓ **Δυνατότητα Επαναχρησιμοποίησης (Reusability)**: Ένα πρόγραμμα είναι επαναχρησιμοποιήσιμο εάν μπορεί **να χρησιμοποιηθεί** (ενδεχομένως με μικρές αλλαγές) για την **ανάπτυξη άλλου προϊόντος λογισμικού** (λ.χ., λογισμικό διαχείρισης αποθεμάτων).
- ✓ **Φορητότητα (Portability)**: Το λογισμικό είναι φορητό εάν μπορεί **να εκτελεστεί σε διαφορετικά περιβάλλοντα** (λειτουργικά συστήματα, εξάρτηση από βάσεις).

ΜΕΡΟΣ Γ’

«ΑΝΑΠΤΥΞΗ ΛΟΓΙΣΜΙΚΟΥ»

ΕΡΓΑΣΙΕΣ & ΑΡΜΟΔΙΟΤΗΤΕΣ

- **Πελάτης**: εταιρεία / οργανισμός / ή πρόσωπο που χρηματοδοτεί το έργο.
- **Χρήστες**: πρόσωπα που θα χρησιμοποιήσουν το λογισμικό.
- **Επωφελούμενοι**: αποδέκτες των υπηρεσιών λογισμικού.
- **Ομάδα Ανάπτυξης**: ομάδα αναλυτών, σχεδιαστών, προγραμματιστών και ελεγκτών.
- **Ομάδα Διοίκησης του Έργου**: λήψη αποφάσεων για τους πόρους του έργου.
- **Ομάδα Διασφάλισης Ποιότητας**: παρακολουθεί την εξέλιξη του έργου και εισηγείται στη διοίκηση του έργου ενέργειες που διασφαλίζουν την ποιότητα του έργου.

ΔΙΟΙΚΗΣΗ ΕΡΓΟΥ

- Ένα έργο ανάπτυξης **δεν περιορίζεται σε καθαρά <u>τεχνικά θέματα</u>**, αλλά και σε άλλα όπως:
  - **ο χρονοπρογραμματισμός του έργου**
  - **η κατάρτιση του προϋπολογισμού του**
  - **η στελέχωση της ομάδας ανάπτυξης**
  - **η οργάνωση της ομάδας ανάπτυξης**
  - **η παρακολούθηση της πορείας του έργου**
  - **η διαχείριση των κινδύνων**
- Όλες αυτές η δραστηριότητες αφορούν τη **<u>διοίκηση του έργου</u>** που είναι εξίσου σημαντικές όπως και οι βασικές.
- Ο ρόλος του διοικητή έργου είναι κρίσιμος για την **<u>επιτυχή ολοκλήρωση</u>** των έργων.

ΤΥΠΙΚΗ ΔΙΑΔΙΚΑΣΙΑ ΑΝΑΠΤΥΞΗΣ ΛΟΓΙΣΜΙΚΟΥ

1. **Προδιαγραφές Απαιτήσεων – Ανάλυση**: Η φάση όπου καθορίζονται οι **ανάγκες** του χρήστη και οι **λειτουργίες** που πρέπει να έχει το λογισμικό.
2. **Σχεδίαση**: Η δημιουργία του **σχεδίου** του λογισμικού, συμπεριλαμβανομένης της **αρχιτεκτονικής**, της **διεπαφής** χρήστη και της **βάσης** δεδομένων.
3. **Ανάπτυξη Κώδικα**: Η **συγγραφή του κώδικα** του λογισμικού, σύμφωνα με τις προδιαγραφές και το σχέδιο.
4. **Επαλήθευση**: Ο έλεγχος του λογισμικού για την **ανίχνευση** και **διόρθωση σφαλμάτων**, καθώς και για την εξασφάλιση της ποιότητας και της λειτουργικότητάς του.
5. **Συντήρηση**: Η συνεχής **υποστήριξη του λογισμικού** μετά την κυκλοφορία του, συμπεριλαμβανομένων των ενημερώσεων, των διορθώσεων και της βελτίωσης των λειτουργιών του.

ΓΙΑ ΤΗ ΔΙΑΧΕΙΡΙΣΗ ΤΗΣ ΑΠΑΙΤΕΙΤΑΙ ΑΚΡΙΒΗΣ ΜΟΝΤΕΛΟΠΟΙΗΣΗ !

ΔΙΑΣΦΑΛΙΣΗ ΠΟΙΟΤΗΤΑΣ

- «Σκοπός της διασφάλισης ποιότητας είναι να **επιβεβαιώνει διαρκώς ότι κάθε παραδοτέο της ομάδας ανάπτυξης διαθέτει το επίπεδο ποιότητας που έχει ορίσει η διοίκηση του έργου** και να **εισηγείται τυχόν αναγκαίες ενέργειες για την βελτίωση της ποιότητας των παραδοτέων** της ομάδας ανάπτυξης».
- Η **επένδυση στην ποιότητα** κατά την ανάπτυξη λογισμικού μπορεί να οδηγήσει σε **μείωση του συνολικού χρόνου ολοκλήρωσης** του έργου, καθώς **η πρόληψη σφαλμάτων και η αποφυγή επαναλήψεων εργασιών εξοικονομεί χρόνο μακροπρόθεσμα.**
- Σε ορισμένες περιπτώσεις, η επίτευξη **εξαιρετικά υψηλών επιπέδων ποιότητας**, ειδικά σε **πολύπλοκα έργα ή με αυστηρές απαιτήσεις**, μπορεί να απαιτήσει **πρόσθετο χρόνο ανάπτυξης λόγω της ανάγκης για εκτενέστερες δοκιμές, αναλύσεις και βελτιστοποιήσεις.**

ΜΕΡΟΣ Δ'

ΜΟΝΤΕΛΑ ΑΝΑΠΤΥΞΗΣ ΛΟΓΙΣΜΙΚΟΥ

ΤΙ ΕΙΝΑΙ ΤΟ ΜΟΝΤΕΛΟ ΑΝΑΥΠΤΥΞΗΣ ΛΟΓΙΣΜΙΚΟΥ;

«Ένα **πλαίσιο** που περιγράφει τις **φάσεις και τις δραστηριότητες** που εμπλέκονται στην **ανάπτυξη και συντήρηση** ενός λογισμικού, από την **σύλληψη** της ιδέας μέχρι την **απόσυρσή** του».

ΠΟΙΟΣ ΕΙΝΑΙ Ο ΣΚΟΠΟΣ ΤΟΥ;

- Η **οργάνωση και διαχείριση** της διαδικασίας ανάπτυξης.
- Η **βελτίωση** της **ποιότητας** του λογισμικού.
- Η **ελαχιστοποίηση** των **κινδύνων** και του κόστους.
- Η **Εξασφάλιση** της έγκαιρης **παράδοσης**.

ΤΙ ΚΑΘΟΡΙΖΕΙ ΕΝΑ ΜΟΝΤΕΛΟ ΚΥΚΛΟΥ ΖΩΗΣ ΛΟΓΙΣΜΙΚΟΥ;

Κάθε μοντέλο **απαντά σε συγκεκριμένα ερωτήματα** για τη διαδικασία ανάπτυξης:

- Πώς δουλεύουμε; Ποια είναι η σειρά των εργασιών; Υπάρχει ευελιξία για αλλαγές;
- Τι κάνουμε σε κάθε βήμα; Ποιες είναι οι συγκεκριμένες δραστηριότητες που πρέπει να ολοκληρωθούν;
- Ποια είναι τα παραδοτέα; Τι έγγραφα, σχέδια ή κομμάτια κώδικα παράγουμε σε κάθε φάση;
- Πότε τελειώνει κάθε φάση; Ποια είναι τα κριτήρια ολοκλήρωσης για να προχωρήσουμε στην επόμενη φάση;

ΟΥΣΙΑΣΤΙΚΑ ΕΙΝΑΙ ΜΙΑ ΔΙΑΔΙΚΑΣΙΑ ΕΠΙΛΥΣΗΣ ΠΡΟΒΛΗΜΑΤΩΝ!

Τα βήματα κάθε τέτοιας διαδικασίας είναι:

1. **Η αποτίμηση τρέχουσας κατάστασης**
2. **Ο ορισμός του προβλήματος**
3. **Η επιλογή μιας λύσης**
4. **Η υλοποίηση της λύσης**
5. **Η ενσωμάτωση της λύσης στο σύστημα**

ΓΙΑΤΙ ΥΠΑΡΧΟΥΝ ΔΙΑΦΟΡΕΤΙΚΑ ΜΟΝΤΕΛΑ;

- **Διαφορετικά έργα, διαφορετικές ανάγκες**

Κάθε έργο έχει τα δικά του χαρακτηριστικά (μέγεθος, πολυπλοκότητα, απαιτήσεις…)

- **Διαφορετικές ομάδες**

Κάθε ομάδα ανάπτυξης έχει τις δικές της προτιμήσεις και δυνατότητες!

- **Διαφορετικές προτεραιότητες**

Μερικές φορές η ταχύτητα είναι πιο σημαντική, άλλες φορές η ποιότητα ή η προσαρμοστικότητα στις αλλαγές είναι πιο σημαντική.

ΜΕΡΟΣ Δ’

«ΜΟΝΤΕΛΑ ΑΝΑΠΤΥΞΗΣ ΛΟΓΙΣΜΙΚΟΥ»

ΤΟ ΜΟΝΤΕΛΟ ΤΟΥ ΚΑΤΑΡΡΑΚΤΗ (WATERFALL)

- **Βασική Αρχή**: Γραμμική, διαδοχική προσέγγιση, ανάπτυξης λογισμικού.
- **Χαρακτηριστικά**
  - Διακριτές, προκαθορισμένες, φάσεις.
  - Κάθε φάση ολοκληρώνεται πλήρως πριν την έναρξη της επόμενης.
  - Κάθε φάση παράγει συγκεκριμένα παραδοτέα.
- **Σημείωση:** Αν και είναι το **παλαιότερο μοντέλο**, παραμένει χρήσιμο σε συγκεκριμένα περιβάλλοντα όπου οι **απαιτήσεις είναι σταθερές** και οι **αλλαγές ελάχιστες**.

ΤΟ ΜΟΝΤΕΛΟ ΤΟΥ ΚΑΤΑΡΡΑΚΤΗ (WATERFALL)

- **Πλεονεκτήματα**
  - Απλό και εύκολα κατανοητό.
  - Κατάλληλο για μικρά έργα με σαφείς απαιτήσεις.
- **Μειονεκτήματα**
  - Δυσκολία προσαρμογής σε αλλαγές απαιτήσεων.
  - Προβλήματα εντοπίζονται αργά, αυξάνοντας το κόστος διόρθωσης.
  - Ο πελάτης δεν βλέπει αποτελέσματα μέχρι το τέλος του έργου.



Σχήμα: Το μοντέλο του καταρράκτη



ΤΟ ΜΟΝΤΕΛΟ ΠΡΩΤΟΤΥΠΟΠΟΙΗΣΗΣ (PROTOTYPING MODEL)

- **Βασική Αρχή: Επαναληπτική και εξελικτική προσέγγιση** ανάπτυξης λογισμικού, όπου η δημιουργία **πρωτοτύπων** οδηγεί στη σταδιακή βελτίωση και τελική ολοκλήρωση του προϊόντος.
- **Χαρακτηριστικά:**
  - Ενεργή συμμετοχή του χρήστη
  - Δημιουργία και βελτίωση πρωτοτύπων
  - Συνεχής ανατροφοδότηση και προσαρμογή
  - Ευελιξία και προσαρμοστικότητα
  - Μείωση του ρίσκου
- **Σημείωση:** Το μοντέλο πρωτοτυποποίησης είναι ιδιαίτερα χρήσιμο σε έργα όπου οι **απαιτήσεις είναι ασαφείς ή μεταβαλλόμενες**, καθώς και σε περιπτώσεις όπου η **καινοτομία** και η **δημιουργικότητα** είναι σημαντικές.



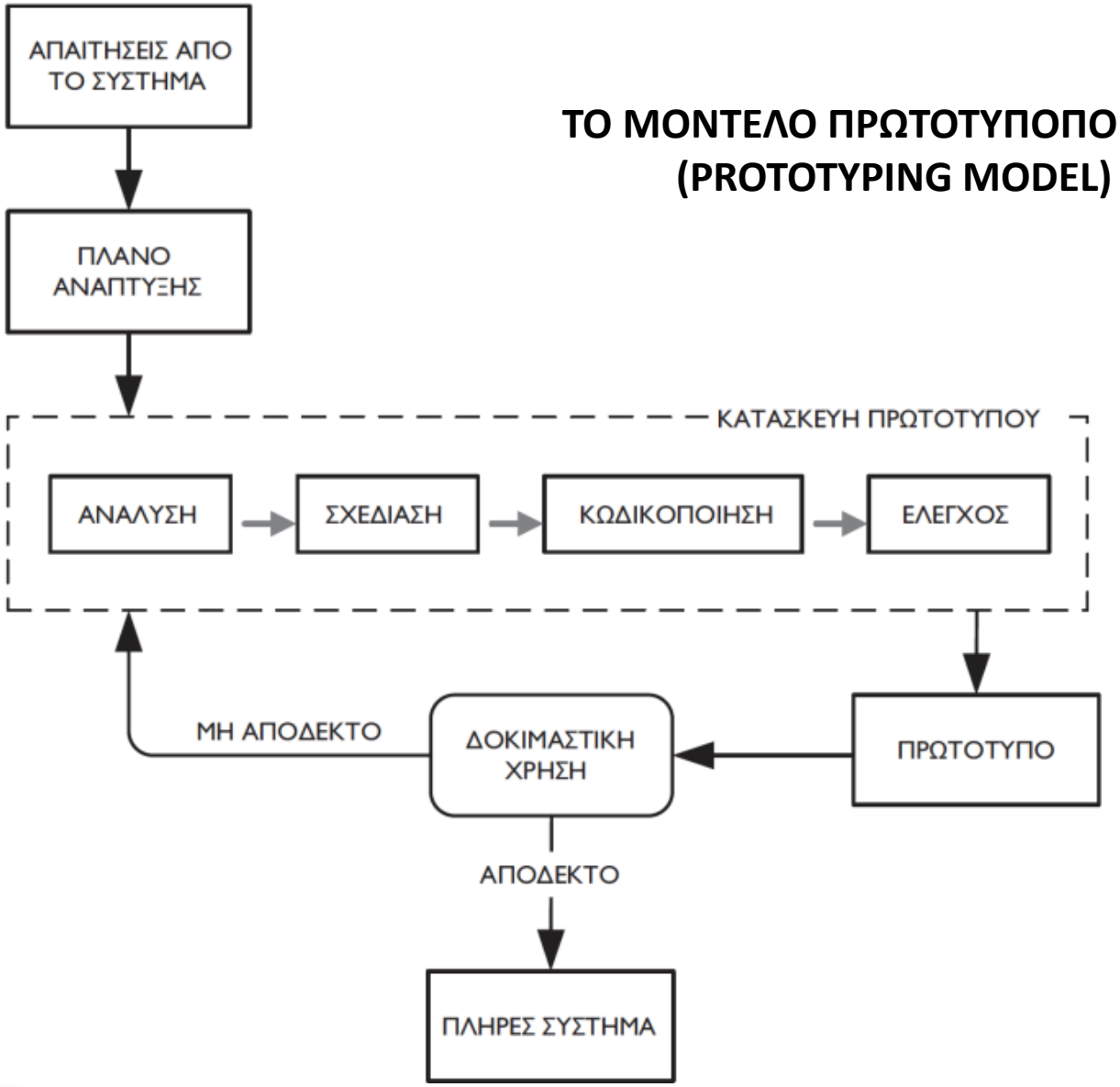
ΤΟ ΜΟΝΤΕΛΟ ΠΡΩΤΟΤΥΠΟΠΟΙΗΣΗΣ (PROTOTYPING MODEL)

Πλεονεκτήματα

- Αυξημένη ικανοποίηση του χρήστη
- Καλύτερη κατανόηση των απαιτήσεων
- Έγκαιρη ανίχνευση προβλημάτων
- Ευελιξία στην αντιμετώπιση αλλαγών

- **Μειονεκτήματα**
  - Αυξημένο κόστος και χρόνος
  - Δυσκολία στον έλεγχο του εύρους του έργου
  - Κίνδυνος προσκόλλησης στο πρωτότυπο
  - Απαιτείται ενεργή συμμετοχή του χρήστη

ΤΟ ΜΟΝΤΕΛΟ ΠΡΩΤΟΤΥΠΟΠΟΙΗΣΗΣ (PROTOTYPING MODEL)



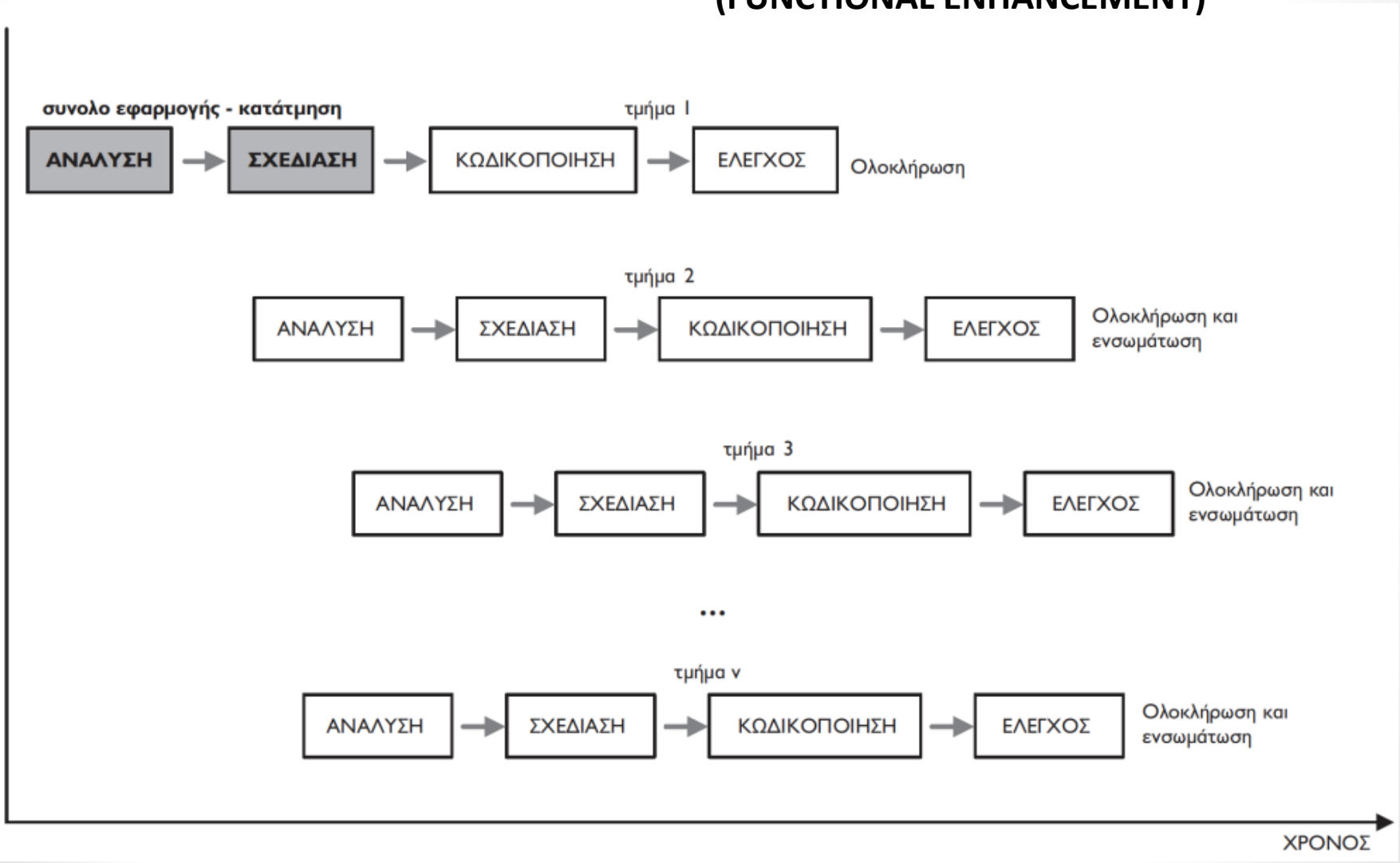
ΤΟ ΜΟΝΤΕΛΟ ΛΕΙΤΟΥΡΓΙΚΗΣ ΕΠΑΥΞΗΣΗΣ (FUNCTIONAL ENHANCEMENT)

- **Βασική Αρχή: Σταδιακή ανάπτυξη** του λογισμικού σε **διακριτές επαυξήσεις**, όπου κάθε επαύξηση **προσθέτει νέες λειτουργίες** και **βελτιώσεις** στο υπάρχον σύστημα.
- **Χαρακτηριστικά:**
  - Καταμερισμός σε επαυξήσεις
  - Ακολουθιακή ανάπτυξη επαυξήσεων
  - Παράδοση σε στάδια
  - Συνεχής βελτίωση
- **Σημείωση:** Το μοντέλο λειτουργικής επαύξησης είναι κατάλληλο για έργα όπου **οι απαιτήσεις είναι γνωστές σε γενικές γραμμές**, αλλά ενδέχεται να αλλάξουν με την πάροδο του χρόνου. Είναι επίσης χρήσιμο όταν υπάρχει ανάγκη για **ταχεία παράδοση λειτουργικότητας** στον χρήστη.

ΤΟ ΜΟΝΤΕΛΟ ΛΕΙΤΟΥΡΓΙΚΗΣ ΕΠΑΥΞΗΣΗΣ (FUNCTIONAL ENHANCEMENT)

- **Πλεονεκτήματα**
  - Ταχύτερη παράδοση αξίας στον χρήστη
  - Ευελιξία στην αντιμετώπιση αλλαγών
  - Μείωση του ρίσκου
  - Βελτιωμένη διαχείριση του έργου
- **Μειονεκτήματα**
  - Απαιτείται προσεκτικός σχεδιασμός
  - Δυσκολία στην εκτίμηση του συνολικού κόστους και χρόνου
  - Κίνδυνος ασυμβατότητας μεταξύ επαυξήσεων
  - Απαιτείται συνεχής επικοινωνία με τον χρήστη

ΤΟ ΜΟΝΤΕΛΟ ΛΕΙΤΟΥΡΓΙΚΗΣ ΕΠΑΥΞΗΣΗΣ (FUNCTIONAL ENHANCEMENT)





ΤΟ ΣΠΕΙΡΟΕΙΔΕΣ ΜΟΝΤΕΛΟ (SPIRAL MODEL)

- **Βασική Αρχή:** Επαναληπτική και εξελικτική προσέγγιση, με έμφαση στην ανάλυση κινδύνων και τη διαχείριση.
- **Χαρακτηριστικά:**
  - **Επαναλαμβανόμενοι κύκλοι ανάπτυξης** με κάθε κύκλο να προσθέτει νέα λειτουργικότητα και βελτιώνει το προϊόν.
  - **Η ανάλυση κινδύνων** γίνεται πριν και κατά τη διάρκεια κάθε κύκλου ανάπτυξης.
  - Οι **λεπτομέρειες υλοποίησης καθορίζονται σταδιακά** σε κάθε κύκλο, επιτρέποντας την προσαρμογή στις μεταβαλλόμενες απαιτήσεις.



ΤΟ ΣΠΕΙΡΟΕΙΔΕΣ ΜΟΝΤΕΛΟ (SPIRAL MODEL)

Κατηγορίες εργασιών (4) σε κάθε κύκλο:

1. **Προσδιορισμός Στόχων:** Καθορισμός των στόχων, των απαιτήσεων και των περιορισμών του τρέχοντος κύκλου.
2. **Εντοπισμός και Επίλυση Κινδύνων:** Αναγνώριση, αξιολόγηση και διαχείριση των πιθανών κινδύνων που σχετίζονται με το έργο.
3. **Εκτέλεση Διαδικασιών Ανάπτυξης και Επαλήθευση:** Σχεδιασμός, υλοποίηση, δοκιμή και ενσωμάτωση των νέων λειτουργιών.
4. **Προγραμματισμός Επόμενης Φάσης:** Αξιολόγηση των αποτελεσμάτων του κύκλου και προγραμματισμός του επόμενου κύκλου.

Σημείωση: Το Σπειροειδές Μοντέλο είναι κατάλληλο για **μεγάλα και σύνθετα έργα**, όπου οι **απαιτήσεις** ενδέχεται να **αλλάζουν** κατά τη διάρκεια της ανάπτυξης ή όταν η διαχείριση κινδύνων είναι κρίσιμη.



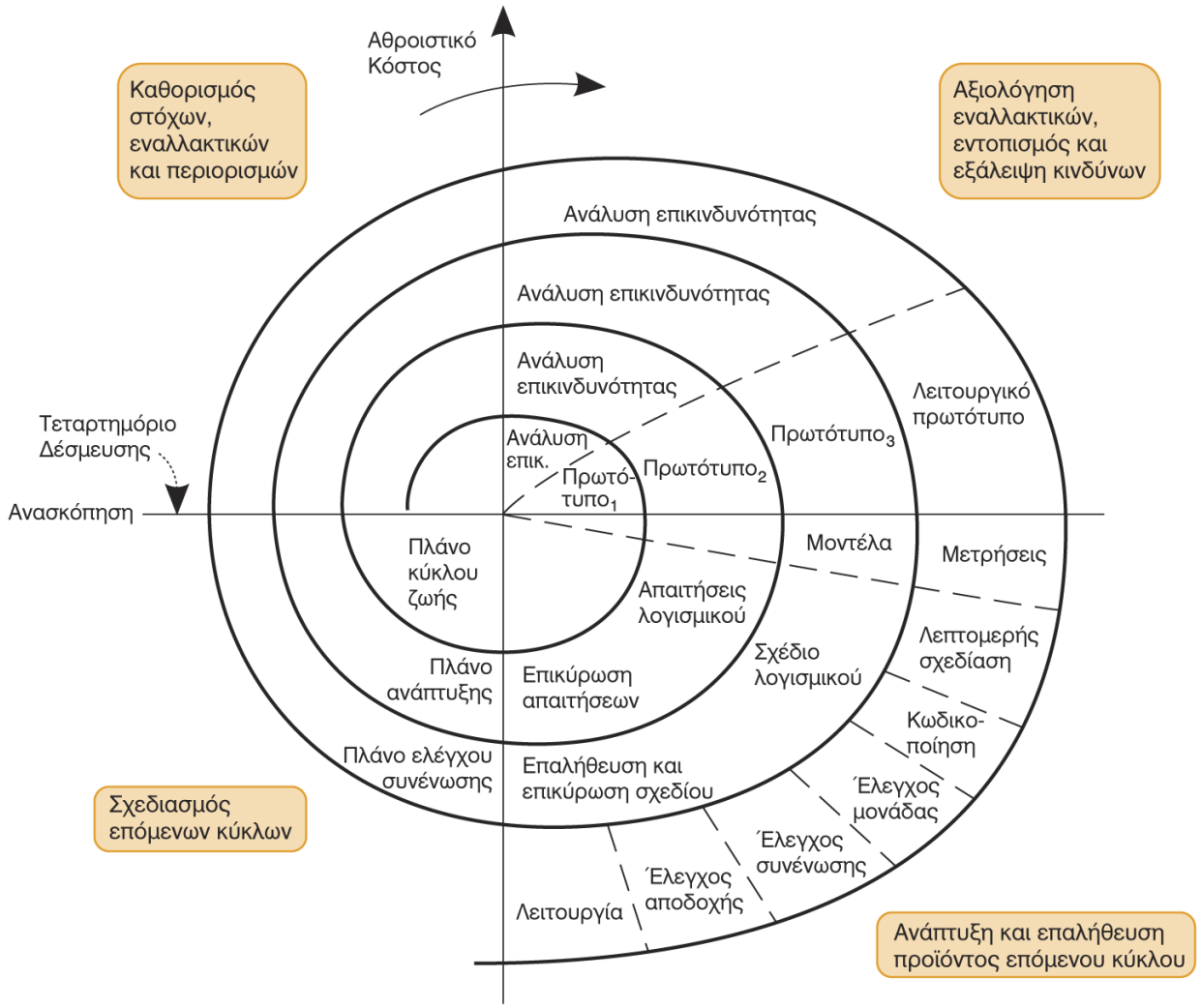
ΤΟ ΣΠΕΙΡΟΕΙΔΕΣ ΜΟΝΤΕΛΟ (SPIRAL MODEL)

Πλεονεκτήματα

- Ελαχιστοποιεί τους κινδύνους μέσω της **συνεχούς αξιολόγησης και προσαρμογής**.
- **Προσαρμόζεται εύκολα** στις μεταβαλλόμενες απαιτήσεις και τις συνθήκες του έργου.
- Επιτρέπει την **σταδιακή παράδοση λειτουργικότητας**, παρέχοντας αξία νωρίτερα στον πελάτη.

Μειονεκτήματα

- Απαιτεί **προσεκτικό σχεδιασμό και διαχείριση** λόγω της ευέλικτης φύσης του.
- Μπορεί να είναι **πιο δαπανηρό από άλλα μοντέλα** λόγω των πολλαπλών κύκλων ανάπτυξης.
- Απαιτεί **έμπειρη ομάδα ανάπτυξης** για την αποτελεσματική εφαρμογή του.



Σχήμα 1-12: Το σπειροειδές μοντέλο

ΣΥΓΚΡΙΣΗ ΜΟΝΤΕΛΩΝ ΚΥΚΛΟΥ ΖΩΗΣ ΛΟΓΙΣΜΙΚΟΥ

(κάπως υποκειμενική)

Μοντέλο	Μέγεθος εφαρμογών	Μεταβολές στις απαιτήσεις
Καταρράκτη	Μικρό έως μεσαίο	Ανεπιθύμητες
Πρωτοτυποποίησης	Μικρό ως μεσαίο	Δεκτές
Λειτουργικής επαύξησης	Μεσαίο ως μεγάλο	Ανεπιθύμητες
Σπειροειδές	Μεσαίο ως μεγάλο	Δεκτές

Η ΕΠΙΛΟΓΗ ΤΟΥ ΚΑΤΑΛΛΗΛΟΥ ΜΟΝΤΕΛΟΥ ΑΝΑΠΤΥΞΗΣ ΛΟΓΙΣΜΙΚΟΥ ΕΙΝΑΙ ΜΙΑ ΑΠΟΦΑΣΗ ΜΕ ΠΟΛΛΕΣ ΠΑΡΑΜΕΤΡΟΥΣ.

Δεν υπάρχει "**one size fits all**" λύση! Η βέλτιστη επιλογή εξαρτάται από το **συγκεκριμένο project και τις ανάγκες του**.

Είναι σημαντικό να θυμόμαστε ότι:

- Κάθε μοντέλο έχει τα δικά του **πλεονεκτήματα** και **μειονεκτήματα**.
- Η επιλογή του μοντέλου επηρεάζει άμεσα την **οργάνωση**, την **επικοινωνία**, και την **αποτελεσματικότητα** της ομάδας.

Ο στόχος είναι να επιλέξουμε το μοντέλο που θα μας βοηθήσει να παραδώσουμε ένα **ποιοτικό προϊόν**, **εντός χρονοδιαγράμματος** και **προϋπολογισμού**, **ικανοποιώντας** τις απαιτήσεις του **πελάτη**.

End

Τι κρατάμε από τη σημερινή διάλεξη

Λογισμικό	Κώδικας, δεδομένα, ρυθμίσεις και τεκμηρίωση.
Ανάπτυξη	Απαιτήσεις, σχεδίαση, υλοποίηση, έλεγχος και συντήρηση.
Ποιότητα	Ορθότητα, αξιοπιστία, αποδοτικότητα και ευχρηστία.
Συνεργασία	Σαφείς ρόλοι, συντονισμός και κοινές αποφάσεις.
Μοντέλο	Επιλογή με βάση τις ανάγκες, τις αλλαγές και τους κινδύνους.

Πρώτο παραδοτέο: πρόταση έργου

Μία σελίδα PDF ανά ομάδα.

Πρόβλημα	Ποια ανάγκη καλύπτει η εφαρμογή και ποιοι θα τη χρησιμοποιούν;
Λειτουργίες	Περιγράψτε 3–5 βασικές λειτουργικές απαιτήσεις.
Ποιότητα	Ορίστε μία απαίτηση ποιότητας, όπως ευχρηστία ή χρόνος απόκρισης.
Ομάδα	Καταγράψτε τα μέλη και μια αρχική κατανομή ρόλων.

Προθεσμία και υποβολή: ανακοίνωση στην ηλεκτρονική τάξη.

Προτεινόμενη Βιβλιογραφία:

1. **Τεχνολογία λογισμικού**, *Γιακουμάκης Μανόλης, Διαμαντίδης Νίκος*
2. **ΕΙΣΑΓΩΓΗ ΣΤΗ UML: ΣΥΝΟΠΤΙΚΟΣ ΟΔΗΓΟΣ ΤΗΣ ΠΡΟΤΥΠΗΣ ΓΛΩΣΣΑΣ ΜΟΝΤΕΛΟΠΟΙΗΣΗΣ ΑΝΤΙΚΕΙΜΕΝΩΝ**, *MARTIN FOWLER*



ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΣΧΟΛΗ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

Εισαγωγή στην Τεχνολογία Λογισμικού.

Thanks for Watching

i.stylios@uoi.gr